

Terraform Stages Guide

This document outlines how Terraform stages are utilized in the development and CI/CD pipeline of your dynamic web hosting project.

1. The Terraform Stage in Jenkinsfile

Your automation setup contains exactly one dedicated stage for executing Terraform deployments:

- **stage('Terraform Apply'):** - Executes code checkout, builds/pushes the Docker image, and triggers infrastructure changes.
- **Credentials Injection:** - Safely passes secret AWS credentials securely as environment variables.
- **State Backup:** - Backs up the local state file so it remains synchronized between the host system and the Jenkins environment.

2. Terraform Lifecycle Commands Used

Within the automated pipeline stage, the following commands are run sequentially:

- **terraform init:** - Downloads provider plugins (specifically HashiCorp AWS) and initializes state storage.
- **terraform apply -auto-approve:** - Deploys all network, security, and computing resources automatically without requiring human yes/no prompt.
- **terraform state rm:** - Executed dynamically in case of resource conflict to delete duplicated Elastic IP allocations.
- **terraform output:** - Reads outputs (like the EC2 public IP address) to deploy the app container.

3. Standard Terraform Lifecycle Stages

In a generic Terraform workflow, there are four standard conceptual stages:

- **Init:** - Initialize the backend configuration and local workspace.
- **Plan:** - Preview the actions that Terraform will perform on the provider's API.
- **Apply:** - Execute the changes to bring resources to the desired state.
- **Destroy:** - Delete all managed infrastructure safely.